

Guide de Déploiement

ONCF Zero-Click Search Recommender

À l'attention de l'équipe SI ONCF

Omar Chekroun – Mai 2026

Table des matières

Objet du document

Ce guide explique comment déployer le service de recommandation *Zero-Click Search* sur l'infrastructure ONCF. Deux voies sont documentées : **Docker** (recommandée) et **Python venv** (fallback si Docker n'est pas disponible).

1 Pré-requis

1.1 Hardware minimal

Ressource	Recommandation
CPU	4 cœurs minimum (8 recommandés pour trafic > 100 req/s)
RAM	8 Go minimum (le modèle XGBoost charge 281 Mo, le lookup historique 250 Mo)
Disque	5 Go pour les artefacts modèles, logs et données traitées
Réseau	accès sortant uniquement si scraper <code>oncf.ma</code> activé
GPU	non requis (l'inférence se fait sur CPU via ONNX Runtime)

1.2 Système d'exploitation

Le service tourne sur :

- **Linux** (Ubuntu 22.04 LTS, Debian 12, RHEL 9) — voie nominale
- **Windows Server 2019/2022** — supporté via Docker Desktop ou WSL2

1.3 Logiciels requis

Pour la voie Docker (recommandée) :

- Docker Engine ≥ 24.0
- Docker Compose v2 (`docker compose`, plugin intégré)

Pour la voie venv (fallback) :

- Python 3.12.x (3.12.10 testé)
- `pip` ≥ 23.0

— Optionnellement Redis 7.x pour le cache des horaires

2 Voie nominale : déploiement par Docker

2.1 Récupération du code

```
git clone <url-du-repo-ONCF> oncf-recommender
cd oncf-recommender
```

2.2 Configuration des variables d'environnement

Copier le fichier d'exemple et l'adapter :

```
cp deploy/.env.example deploy/.env
nano deploy/.env # ou vi/notepad
```

Variables documentées :

Variable	Rôle
LOG_LEVEL	Niveau de log (DEBUG, INFO, WARNING, ERROR)
MODEL_DIR	Chemin vers les artefacts modèles (par défaut /app/models)
DATA_DIR	Chemin vers les datasets traités (par défaut /app/data/processed)
REDIS_URL	URL Redis pour le cache horaires (ex. redis://redis:6379/0)
API_PORT	Port d'écoute sur l'hôte (par défaut 8000)
API_WORKERS	Nombre de workers uvicorn (1 suffit pour la charge ONCF typique)

2.3 Placement des artefacts modèles

Les modèles ne sont **pas** embarqués dans l'image Docker pour faciliter les promotions sans rebuild. Ils sont montés en volume depuis l'hôte :

```
# Depuis la racine du projet
ls models/
# Attendu:
# xgb_ranker.json          (281 Mo)
# xgb_ranker.onnx          (148 Mo)
# label_encoder.joblib     (8.5 Ko)
# cold_start.joblib        (31 Ko)
# popularity.joblib        (~120 Ko)
# xgb_ranker.meta.json     (~2 Ko)
```

Si `models/` est absent ou incomplet, transférer les artefacts depuis l'environnement de pré-production. Lancer le pipeline complet de ré-entraînement (`scripts/03_train_ranker.py`, `05_build_cold_start.py`, `06_export_onnx.py`, `08_build_popularity.py`) prend environ 50 minutes sur CPU.

2.4 Build et démarrage

```
cd deploy
docker compose build
docker compose up -d
```

Vérification du healthcheck :

```
docker compose ps
# api ... healthy
# redis ... healthy
```

```
curl http://localhost:8000/health
# {"status":"ok","model_loaded":true,"popularity_loaded":true,"n_users_history":69449}
```

2.5 Test fonctionnel

```
curl -X POST http://localhost:8000/recommend \
-H "Content-Type: application/json" \
-d '{"code_client":"12345","k":3}'
```

Réponse attendue (exemple) :

```
{
  "mode": "model",
  "variant": "a",
  "request_id": "...",
  "recommendations": ["LIAISON_1","LIAISON_2","LIAISON_3"],
  "labels": {"LIAISON_1":"CASA -> RABAT", ...}
}
```

2.6 Arrêt et logs

```
docker compose down          # arrêt propre
docker compose logs -f api   # suivi des logs
docker compose logs --tail=200 # 200 dernières lignes
```

3 Voie alternative : Python venv

À utiliser si Docker n'est pas disponible sur l'infrastructure ONCF.

3.1 Création de l'environnement

Linux :

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install fastapi==0.115.* "uvicorn[standard]==0.30.*" \
  "pydantic>=2.0,<3.0" "pandas>=3.0,<4.0" \
  "xgboost==3.2.*" "scikit-learn>=1.4,<2.0" \
  "onnxruntime>=1.18,<2.0" "joblib>=1.4,<2.0" \
```

```
"requests">=2.31,<3.0" "beautifulsoup4">=4.12,<5.0" \
"loguru">=0.7,<1.0" "redis">=5.0,<6.0"
```

Windows :

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install fastapi==0.115.* uvicorn[standard]==0.30.* `
    pydantic pandas xgboost scikit-learn onnxruntime `
    joblib requests beautifulsoup4 loguru redis
```

3.2 Lancement direct (test)

```
# Linux
uvicorn apps.api.main:app --host 0.0.0.0 --port 8000

# Windows
.venv\Scripts\python.exe -m uvicorn apps.api.main:app --host 0.0.0.0 --port 8000
```

3.3 Lancement comme service système

Linux (systemd) : créer /etc/systemd/system/oncf-recommender.service :

```
[Unit]
Description=ONCF Recommender API
After=network.target

[Service]
Type=simple
User=oncf
WorkingDirectory=/opt/oncf-recommender
Environment="PYTHONPATH=/opt/oncf-recommender/src"
ExecStart=/opt/oncf-recommender/.venv/bin/uvicorn apps.api.main:app --host 0.0.0.0 --port 8000
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Activation :

```
sudo systemctl daemon-reload
sudo systemctl enable oncf-recommender
sudo systemctl start oncf-recommender
sudo systemctl status oncf-recommender
```

Windows Server (NSSM) :

```
# Installer NSSM (Non-Sucking Service Manager) au préalable
nssm install ONCFRecommender ^
    "C:\path\to\.venv\Scripts\python.exe" ^
    "-m" "uvicorn" "apps.api.main:app" "--host" "0.0.0.0" "--port" "8000"
nssm set ONCFRecommender AppDirectory "C:\path\to\oncf-recommender"
nssm start ONCFRecommender
```

La voie `venv` ne fournit pas l'isolation réseau, le redémarrage automatique sur crash applicatif, ni le healthcheck standard. Le monitoring devra s'appuyer sur `systemctl status` (Linux) ou le journal des services Windows.

4 Observabilité

4.1 Logs

L'API écrit des logs structurés (JSON) dans `/app/logs/api.log` (Docker) ou `logs/api.log` (venv), avec rotation automatique (10 Mo / fichier, rétention 7 jours).

Champs journalisés à chaque appel `/recommend` : `event`, `variant`, `request_id`, `mode`, `k`, `latency_ms`, `n_recommendations`.

Conformité Loi 09-08

Le `code_client` n'apparaît jamais dans les logs — uniquement l'UUID de requête. Vérifier en production : `grep code_client logs/api.log` doit ne rien retourner.

4.2 Healthcheck enrichi

```
curl http://localhost:8000/health
```

Champ	Signification
<code>status</code>	"ok" si le modèle est chargé, "degraded" sinon
<code>model_loaded</code>	Booléen — session ONNX disponible
<code>popularity_loaded</code>	Booléen — liste de fallback chargée
<code>n_users_history</code>	Taille du lookup historique en mémoire

5 Ré-entraînement périodique

Le pipeline de ré-entraînement (`scripts/07_retrain.py`) inclut un **garde-fou KPI** : un nouveau modèle n'est promu en production que si `HR01` n'a pas baissé de plus de 5 points.

5.1 Lancement manuel (test)

```
# Évaluation seule (sans promotion)
.venv/bin/python scripts/07_retrain.py --dry-run

# Évaluation + promotion si KPI OK
.venv/bin/python scripts/07_retrain.py
```

5.2 Planification

Linux (cron) :

```
0 2 * * * /opt/oncf-recommender/.venv/bin/python /opt/oncf-recommender/scripts/07_retrain.py >>
/opt/oncf-recommender/logs/retrain.log 2>&1
```

Windows : utiliser le fichier `tasks/oncf_daily_retrain.xml` fourni dans le repo :

```
schtasks /Create /XML tasks\oncf_daily_retrain.xml /TN "ONCF\DailyRetrain" /F
```

6 Troubleshooting

6.1 Erreurs fréquentes

1. `RuntimeError: Model not found`

Cause : les fichiers `models/xgb_ranker.json`, `label_encoder.joblib` ou `xgb_ranker.onnx` sont absents du volume monté.

Solution : vérifier la présence des fichiers et leur permission de lecture pour l'UID 1000 (utilisateur `oncf` dans le conteneur).

2. `Redis unavailable --- using in-memory schedule cache`

Cause : Redis n'est pas joignable.

Solution : non bloquant — le fallback en mémoire fonctionne. Pour restaurer Redis, vérifier `docker compose ps redis` et `redis-cli -h redis ping`.

3. Latence p99 anormalement élevée

Cause possible : un seul worker `uvicorn` saturé.

Solution : augmenter `API_WORKERS` dans `.env` (chaque worker charge un modèle en mémoire, prévoir +1 Go RAM par worker).

4. `ImportError: redis`

Cause : la voie `venv` ne l'a pas installé.

Solution : `pip install redis` (optionnel — le code fonctionne sans).

5. PDF du rapport ne compile pas

Hors scope déploiement — contacter le développeur (non critique pour le service).

7 Rollback

7.1 Modèle (cas le plus fréquent)

Le pipeline de ré-entraînement écrit toujours un **challenger** avant de promouvoir :

```
ls models/
# xgb_ranker.json          <-- prod
# xgb_ranker_challenger.json <-- dernier challenger évalué
# xgb_ranker_previous.json <-- ancien prod (si promotion récente)
```

Pour annuler la dernière promotion :

```
cd models/
mv xgb_ranker.json xgb_ranker_bad.json
mv xgb_ranker_previous.json xgb_ranker.json
# idem pour label_encoder et .onnx
docker compose restart api
```

7.2 Code applicatif

```
git log --oneline -10  
git checkout <sha-stable>  
cd deploy && docker compose build api && docker compose up -d api
```